05. 가상 영역 관리 (Virtual Region Management)

목차

1. 개요
2. VirtualRegionManager 트레이트
3. SinglePhaseCoreManager
4. CopyConstraintManager
5. LookupAnyManager
6. 가상-물리 매핑 프로세스
7. 관련 문서

개요

halo2-lib의 핵심 설계 철학 중 하나는 **가상 영역(Virtual Region)** 관리입니다. 이는 회로 작성자가 물리적인 컬럼과 행의 배치를 신경 쓰지 않고 논리적인 연산 흐름에만 집중할 수 있게 해줍니다.

가상 영역이란?

가상 영역은 회로의 실행 추적(execution trace)을 저장하는 논리적 공간입니다. 각 `Context` 는 하나의 가상 컬럼으로 볼 수 있으며, 여러 `Context` 들이 모여 하나의 가상 영역을 형성합니다.

핵심 개념:

- **가상 컬럼**: `Context` 의 `advice` 벡터가 하나의 가상 컬럼
- **물리적 컬럼**: 실제 Halo2 회로의 advice 컬럼

- **매핑**: 가상 컬럼들을 물리적 컬럼들로 변환하는 과정
- **Break Points**: 가상 컬럼을 물리적 컬럼으로 나누는 지점

VirtualRegionManager 트레이트

`VirtualRegionManager` 는 가상 영역을 관리하고 물리적 Halo2 영역으로 할당하는 책임을 가진 트레이트입니다.

트레이트 정의

```
// [소스]: halo2-base/src/virtual_region/manager.rs


pub trait VirtualRegionManager<F: Field> {
/// Halo2 config with associated columns and gates
type Config: Clone;
/// Return type of the `assign_raw` method
type Assignment = ();


/// Assign virtual region to the raw region described by `config`
fn assign_raw(&self, config: &Self::Config, region: &mut Region<F>) -> Self::As
}
```

주요 구성 요소

1. **Config 타입**

- 물리적 Halo2 영역을 설명하는 설정
- 컬럼과 게이트 정보를 포함
- 각 구현체마다 다른 Config 타입 사용

2. **assign_raw() 메서드**

- 가상 영역을 물리적 영역에 할당하는 핵심 메서드
- `config` : 물리적 컬럼 설정
- `region` : Halo2의 물리적 영역
- 반환값: 할당 결과 (기본값은 `()`)

역할

`VirtualRegionManager` 는 다음을 담당합니다:

- 가상 셀들을 물리적 셀로 매핑
- 제약 조건 부과 (keygen 시)
- Witness 값 할당 (proving 시)

SinglePhaseCoreManager

`SinglePhaseCoreManager` 는 단일 challenge phase의 모든 `Context` 들을 관리하는 핵심 매니저입니다.

구조체 정의

```
// [소스]: halo2-base/src/gates/flex_gate/threads/single_phase.rs
```

```
pub struct SinglePhaseCoreManager<F: ScalarField> {
```

```rust
/// Virtual columns (contexts)
pub threads: Vec<Context<F>>,
/// Global shared copy manager
pub copy_manager: SharedCopyConstraintManager<F>,
/// Flag for witness generation only
witness_gen_only: bool,
/// Use unknown values during keygen
use_unknown: bool,
/// Challenge phase
phase: usize,
/// Break points for computation graph
pub break_points: RefCell<Option<ThreadBreakPoints>>,
}
```

가상 컬럼 관리

`SinglePhaseCoreManager` 는 여러 `Context` (threads)를 하나의 논리적 단위로 관리합니다:

```rust
// Context 생성
let mut manager = SinglePhaseCoreManager::new(false, copy_manager);

// 메인 스레드 가져오기
let ctx = manager.main();
```

```
// 새 스레드 생성

let  ctx2 = manager.new_thread();
```

각 `Context` 는 독립적인 가상 컬럼이지만, 모두 같은 phase에 속합니다.

물리적 컬럼으로 Chunking

`SinglePhaseCoreManager` 의 핵심 기능은 가상 컬럼들을 물리적 컬럼들로 "chunking"하는 것입니다.

Chunking 과정:

1. **가상 컬럼 연결**: 모든 `Context` 의 advice 벡터를 순차적으로 연결
2. **Break Points 계산**: 물리적 컬럼의 최대 행 수(`usable_rows`)를 고려하여 분할 지점 결정
3. **물리적 할당**: 각 chunk를 별도의 물리적 컬럼에 할당

```
가상 컬럼들:

Context 0: [a0, a1, a2, ..., a99] (100 cells)

Context 1: [b0, b1, b2, ..., b49] (50 cells)

Context 2: [c0, c1, c2, ..., c79] (80 cells)



연결된 가상 컬럼: [a0...a99, b0...b49, c0...c79] (230 cells)



usable_rows = 100이라면:

물리적 컬럼 0: [a0...a99] (break point at 99)

물리적 컬럼 1: [b0...b49, c0...c49] (break point at 149)

물리적 컬럼 2: [c50...c79]
```

Break Points 활용

Break points는 가상 컬럼을 물리적 컬럼으로 나누는 지점을 나타냅니다:

```
// Break points 설정 (proving 시)
manager.break_points.borrow_mut() = Some(vec![99, 149]);


// Break points 사용 (keygen 시 자동 계산)
let break_points = assign_with_constraints(
&manager.threads,
config,
region,
&mut copy_manager,
usable_rows,
use_unknown,
);
```

Break point에서의 특수 처리:

- 게이트가 여러 행을 사용하는 경우 (예: ROTATIONS = 4)
- Break point에서 overlap이 발생할 수 있음
- 현재 셀을 다음 컬럼의 첫 행에 복사하여 연속성 보장

CopyConstraintManager

`CopyConstraintManager` 는 모든 equality constraint를 전역적으로 관리합니다.

구조체 정의

```rust
// [소스]: halo2-base/src/virtual_region/copy_constraints.rs


pub struct CopyConstraintManager<F: Field + Ord> {
/// Equality constraints between advice cells
pub advice_equalities: Vec<(ContextCell, ContextCell)>,
/// Equality constraints between constants and advice cells
pub constant_equalities: Vec<(F, ContextCell)>,
/// Assigned advice cells (virtual -> physical)
pub assigned_advices: HashMap<ContextCell, Cell>,
/// Assigned constant cells
pub assigned_constants: BTreeMap<F, Cell>,
}
```

Advice 셀 간 Equality

`advice_equalities` 벡터는 두 advice 셀이 같은 값을 가져야 한다는 제약을 저장합니다:

```rust
// Context에서 equality constraint 추가
ctx.constrain_equal(&a, &b);


// 내부적으로 CopyConstraintManager에 추가됨
copy_manager.advice_equalities.push((a.cell.unwrap(), b.cell.unwrap()));
```

사용 시나리오:

- `QuantumCell::Existing` 을 사용할 때 자동으로 추가
- 명시적으로 `constrain_equal()` 호출 시

Constant와 Advice 간 Equality

`constant_equalities` 벡터는 advice 셀이 특정 상수 값을 가져야 한다는 제약을 저장합니다:

```
// Constant 로드
let c = ctx.load_constant(F::from(42));

// 내부적으로 CopyConstraintManager에 추가됨
copy_manager.constant_equalities.push((F::from(42), c.cell.unwrap()));
```

Equality Constraint 부과 과정

`CopyConstraintManager` 의 `assign_raw()` 메서드는 모든 equality constraint를 실제로 부과합니다:

```
// [소스]: halo2-base/src/virtual_region/copy_constraints.rs

impl<F: Field + Ord> VirtualRegionManager<F> for SharedCopyConstraintManager<F>
type Config = Vec<Column<Fixed>>;

fn assign_raw(&self, config: &Self::Config, region: &mut Region<F>) {
let mut manager = self.lock().unwrap();
```

```
// 1. Constant 할당
manager.constant_equalities.par_sort_unstable();
let mut fixed_col = 0;
let mut fixed_offset = 0;
for (c, _) in &manager.constant_equalities {
if !manager.assigned_constants.contains_key(c) {
let cell = raw_assign_fixed(
region,
config[fixed_col],
fixed_offset,
*c
);
manager.assigned_constants.insert(*c, cell);
fixed_col += 1;
if fixed_col >= config.len() {
fixed_col = 0;
fixed_offset += 1;
}
}
}
// 2. Advice equality constraints
for (left, right) in &manager.advice_equalities {
let left_cell = manager.assigned_advices[left];
```

```
let right_cell = manager.assigned_advices[right];
raw_constrain_equal(region, left_cell, right_cell);
}
// 3. Constant equality constraints
for (constant, advice) in &manager.constant_equalities {
let const_cell = manager.assigned_constants[constant];
let advice_cell = manager.assigned_advices[advice];
raw_constrain_equal(region, const_cell, advice_cell);
}
}
}
```

중요 사항:

- `CopyConstraintManager` 는 다른 모든 매니저가 셀을 할당한 **후**에 실행되어야 함
- Thread-safe: `Arc<Mutex<>>` 로 감싸져 있어 여러 스레드에서 안전하게 사용 가능

코드 예시

```
// Equality constraint 사용 예시
let gate = GateChip::default();
let ctx = builder.main(0);

// 두 값 할당
let a = ctx.load_witness(F::from(5));
```

```
let b = ctx.load_witness(F::from(5));


// Equality constraint 부과
ctx.constrain_equal(&a, &b);


// 또는 QuantumCell::Existing 사용 시 자동
let c = gate.add(ctx, QuantumCell::Existing(a), QuantumCell::Witness(F::from(3)
// 내부적으로 a와 새 셀 간 equality constraint 생성
```

LookupAnyManager

`LookupAnyManager` 는 lookup argument를 처리하는 매니저입니다.

구조체 정의

```
// [소스]: halo2-base/src/virtual_region/lookups.rs


pub struct LookupAnyManager<F: Field + Ord, const ADVICE_COLS: usize> {
/// Cells to lookup, tagged by (type_id, context_id)
pub cells_to_lookup: Arc<Mutex<BTreeMap<ContextTag, Vec<[AssignedValue<F>; ADVI
/// Global shared copy manager
copy_manager: SharedCopyConstraintManager<F>,
/// Witness generation only flag
witness_gen_only: bool,
```

```
}
```

Lookup Argument 처리 방식

Lookup argument는 특정 값들이 lookup 테이블에 존재함을 증명하는 메커니즘입니다.

처리 과정:

1. **Lookup 요청 추가**

```
// Range check를 위한 lookup
let  range = builder.range_chip();
range.range_check(ctx, value, 64);
// 내부적으로 LookupAnyManager에 추가됨
lookup_manager.add_lookup(ctx.tag(), [value]);
```

2. **특수 Advice 컬럼 사용**

- Lookup이 활성화된 별도의 advice 컬럼 사용
- 이 컬럼들은 항상 lookup constraint가 켜져 있음

3. **값 복사 메커니즘**

- Lookup할 값을 특수 컬럼으로 복사
- Equality constraint로 원본과 복사본이 같음을 보장

assign_raw 구현

```
// [소스]: halo2-base/src/virtual_region/lookups.rs
```

```
impl<F: Field + Ord, const ADVICE_COLS: usize> VirtualRegionManager<F>
for LookupAnyManager<F, ADVICE_COLS>
{
type Config = Vec<[Column<Advice>; ADVICE_COLS]>;


fn  assign_raw(&self, config: &Self::Config, region: &mut Region<F>) {
let  mut  copy_manager = (!self.witness_gen_only)
.then(|| self.copy_manager.lock().unwrap());
let  cells_to_lookup = self.cells_to_lookup.lock().unwrap();
// 왼쪽에서 오른쪽, 위에서 아래로 할당
let  mut  lookup_offset = 0;
let  mut  lookup_col = 0;
for  advices  in  cells_to_lookup.iter().flat_map(|(_, advices)| advices) {
if  lookup_col >= config.len() {
lookup_col = 0;
lookup_offset += 1;
}
for (advice, &column) in  advices.iter().zip(config[lookup_col].iter()) {
let  cell = raw_assign_advice(
region,
column,
lookup_offset,
Value::known(advice.value)
```

```
);
// Equality constraint 추가
if  let Some(cm) = copy_manager.as_mut() {
constrain_virtual_equals_external(
region,
*advice,
cell.cell(),
cm
);
}
}
lookup_col += 1;
}
}
}
```

최적 컬럼 수 계산

```
// 필요한 lookup advice 컬럼 수 계산
let  total_lookups = lookup_manager.total_rows();
let  num_lookup_cols = (total_lookups + usable_rows - 1) / usable_rows;
```

가상-물리 매핑 프로세스

가상 컬럼에서 물리적 컬럼으로의 전체 변환 과정을 살펴봅니다.

전체 프로세스 다이어그램

```
┌──────────────────────────────────────────────────────────────────┐
| 가상 영역 (Virtual Region) |
├──────────────────────────────────────────────────────────────────┤
| |
| Context 0 (thread 0): |
| ┌──────────────────────────────────────────┐ |
| | advice: [a₀, a₁ , a₂ , ..., a₉₉] | |
| | selector: [T, F, T, ..., F] | |
| └──────────────────────────────────────────┘ |
| |
| Context 1 (thread 1): |
| ┌──────────────────────────────────────────┐ |
| | advice: [b₀, b₁ , b₂ , ..., b₄ ₉] | |
| | selector: [F, T, F, ..., T] | |
| └──────────────────────────────────────────┘ |
| |
| Context 2 (thread 2): |
| ┌──────────────────────────────────────────┐ |
```

```
| | advice: [c₀, c₁ , c₂ , ..., c₇₉] | |
| | selector: [T, T, F, ..., F] | |
| └──────────────────────────────────────┘ |
| |
└──────────────────────────────────────────────────────────────┘
↓
SinglePhaseCoreManager
(Break Points 계산)
↓
┌──────────────────────────────────────────────────────────────┐
| 물리적 영역 (Physical Region) |
├──────────────────────────────────────────────────────────────┤
| |
| Column 0: Column 1: Column 2: |
| ┌──────────┐ ┌──────────┐ ┌──────────┐ |
| | a₀ | | b₀ | | c₅₀ | |
| | a₁  | | b₁  | | c₅₁  | |
| | ... | | ... | | ... | |
| | a₉₉ | ← BP | b₄₉ | | c₇₉ | |
| └──────────┘ | c₀ | └──────────┘ |
| | c₁  | |
| | ... | |
| | c₄₉ | ← BP |
```

```
│ └──────────┘ │
│ │
└──────────────────────────────────────────────────────────────┘
```

```
BP = Break Point (물리적 컬럼 전환 지점)
```

Break Points 계산 과정

Break points는 `assign_with_constraints` 함수에서 자동으로 계산됩니다:

```rust
// [소스]: halo2-base/src/gates/flex_gate/threads/single_phase.rs

pub fn assign_with_constraints<F: ScalarField, const ROTATIONS: usize>(
threads: &[Context<F>],
basic_gates: &[BasicGateConfig<F>],
region: &mut Region<F>,
copy_manager: &mut CopyConstraintManager<F>,
max_rows: usize,
use_unknown: bool,
) -> ThreadBreakPoints {
let mut break_points = vec![];
let mut gate_index = 0;
let mut row_offset = 0;
for ctx in threads {
```

```
for (i, (advice, &q)) in  ctx.advice.iter().zip(ctx.selector.iter()).enumerate()

let  basic_gate = basic_gates[gate_index];

let  column = basic_gate.value;

// 셀 할당

let  cell = region.assign_advice(column, row_offset, value).cell();

copy_manager.assigned_advices.insert(

ContextCell::new(ctx.type_id, ctx.context_id, i),

cell

);

// Break point 체크

if (q && row_offset + ROTATIONS > max_rows) || row_offset >= max_rows - 1 {

break_points.push(row_offset);

row_offset = 0;

gate_index += 1;

// 다음 컬럼의 첫 행에 현재 값 복사 (overlap 처리)

let  next_column = basic_gates[gate_index].value;

let  ncell = region.assign_advice(next_column, 0, value).cell();

raw_constrain_equal(region, ncell, cell);

}

// Selector 활성화

if  q {

basic_gate.q_enable.enable(region, row_offset).unwrap();

}
```

```
row_offset += 1;
}
}
break_points
}
```

Break point 발생 조건:

1. Selector가 활성화되어 있고 (`q == true`)
2. 현재 행에서 게이트를 실행하면 `max_rows` 를 초과하는 경우
3. 또는 현재 행이 `max_rows - 1` 에 도달한 경우

ContextCell의 역할

`ContextCell` 은 가상 셀의 위치를 나타내는 포인터입니다:

```
// [소스]: halo2-base/src/lib.rs

pub struct ContextCell {
/// Virtual region type identifier
pub type_id: &'static str,
/// Context identifier
pub context_id: usize,
/// Offset within the context
pub offset: usize,
}
```

매핑 과정에서의 역할:

1. **가상 셀 식별**

```
let virtual_cell = ContextCell::new("halo2-base:SinglePhaseCoreManager:FirstPha
```

2. **물리적 셀로 매핑**

```
let physical_cell = copy_manager.assigned_advices.get(&virtual_cell).unwrap();
```

3. **Equality constraint 부과**

```
let left_virtual = ContextCell::new(type_id, ctx_id, offset1);
let right_virtual = ContextCell::new(type_id, ctx_id, offset2);
let left_physical = copy_manager.assigned_advices[&left_virtual];
let right_physical = copy_manager.assigned_advices[&right_virtual];
raw_constrain_equal(region, left_physical, right_physical);
```

전체 프로세스 코드 예시

다음은 가상 영역 생성부터 물리적 할당까지의 전체 과정입니다:

```
use halo2_base::gates::circuit::BaseCircuitBuilder;
use halo2_base::gates::GateChip;
use halo2_base::utils::ScalarField;

fn virtual_to_physical_example<F: ScalarField>() {
```

```rust
// 1. Circuit builder 생성
let mut builder = BaseCircuitBuilder::new(false);
builder.set_k(10); // 2^10 = 1024 rows
// 2. 가상 영역에서 연산 수행
let gate = GateChip::default();
{
let ctx = builder.main(0);
// 가상 셀들 생성
let a = ctx.load_witness(F::from(5));
let b = ctx.load_witness(F::from(3));
let c = gate.mul(ctx, a, b); // c = a * b = 15
// 더 많은 연산...
for i in 0..100 {
let x = ctx.load_witness(F::from(i));
let y = gate.add(ctx, x, c);
// ...
}
}
// 3. 회로 파라미터 계산 (break points 자동 계산)
builder.calculate_params(Some(9));
// 4. MockProver로 검증 (내부적으로 assign_raw 호출)
use halo2_base::halo2_proofs::dev::MockProver;
MockProver::run(10, &builder, vec![])
```

```
.unwrap()
.assert_satisfied();
// assign_raw가 호출되면:
// - SinglePhaseCoreManager가 가상 컬럼들을 물리적 컬럼으로 매핑
// - Break points에 따라 컬럼 전환
// - CopyConstraintManager가 equality constraints 부과
// - LookupAnyManager가 lookup cells 할당
}
```

최적화 고려사항

1. **컬럼 수 vs 행 수 트레이드오프**

- 더 많은 컬럼 → 더 적은 행 필요 → 더 작은 k
- 더 적은 컬럼 → 더 많은 행 필요 → 더 큰 k

2. **Break point overhead**

- Break point에서 셀 복사 필요 (overlap 처리)
- 너무 많은 break points는 비효율적

3. **병렬 witness 생성**

- 각 Context는 독립적으로 witness 생성 가능
- `BTreeMap` 으로 deterministic ordering 보장

관련 문서

- [02. 핵심 개념](#) - Context, AssignedValue, ContextCell

- 03. 회로 구성 프로세스 - BaseCircuitBuilder 사용법
- 04. 제약 조건 시스템 - FlexGate와 RangeGate

다음 단계

가상 영역 관리를 이해했다면, 이제 타원곡선 암호 연산의 구현을 학습하세요:

→ **06. 타원곡선 암호**: halo2-ecc의 ECC 연산 구현과 최적화 기법

참고 자료

- `halo2-base/src/virtual_region/manager.rs` - VirtualRegionManager 트레이트
- `halo2-base/src/virtual_region/copy_constraints.rs` - CopyConstraintManager 구현
- `halo2-base/src/virtual_region/lookups.rs` - LookupAnyManager 구현
- `halo2-base/src/gates/flex_gate/threads/single_phase.rs` - SinglePhaseCoreManager 구현
- `halo2-base/src/gates/circuit/builder.rs` - BaseCircuitBuilder 구현